Desenvolvimento Web 1

Profa. Ariadne Arrais

O que é o desenvolvimento WEB?

- É a construção de sistemas e aplicações que funcionam através de navegadores.
  - Não é só design;
  - Não é só programação.
  - É a construção da <u>interface</u> + funcionamento de sistemas WEB.

O que é a WEB?

- É um ambiente onde:
  - Usuários acessam aplicações;
  - Sistemas rodam através de navegadores;
  - Computadores trocam informações.

Funcionamento

- Quando você acessa um site:
    1. O navegador faz um pedido;
    2. O servidor responde;
    3. O navegador interpreta os arquivos.

- Um site é um conjunto de arquivos que o navegador “entende”.

Um site é feito de que?

- Um *site* é um conjunto de arquivos:
  - Texto
  - Imagens
  - Código

O que é o desenvolvimento WEB?

- É a área da computação que cria aplicações que funcionam na WEB.

- Inclui:
  - Interface
  - Lógica
  - Dados

Áreas

Usuário ⇄ **Front-end** ⇄ Back-end ⇄ Banco de Dados

(o que vê) (lógica) (dados)

Front-end

- Interface
- Layout
- Interação

Por que isso é importante?

- Front-end é usado em:
  - Sistemas empresariais
  - Aplicações móveis (interfaces)
  - Plataformas educacionais

Mas onde esse sistema roda?

- Desenvolvimento Web cria sistemas
- Esses sistemas rodam no navegador
- Existe front-end, back-end e dados

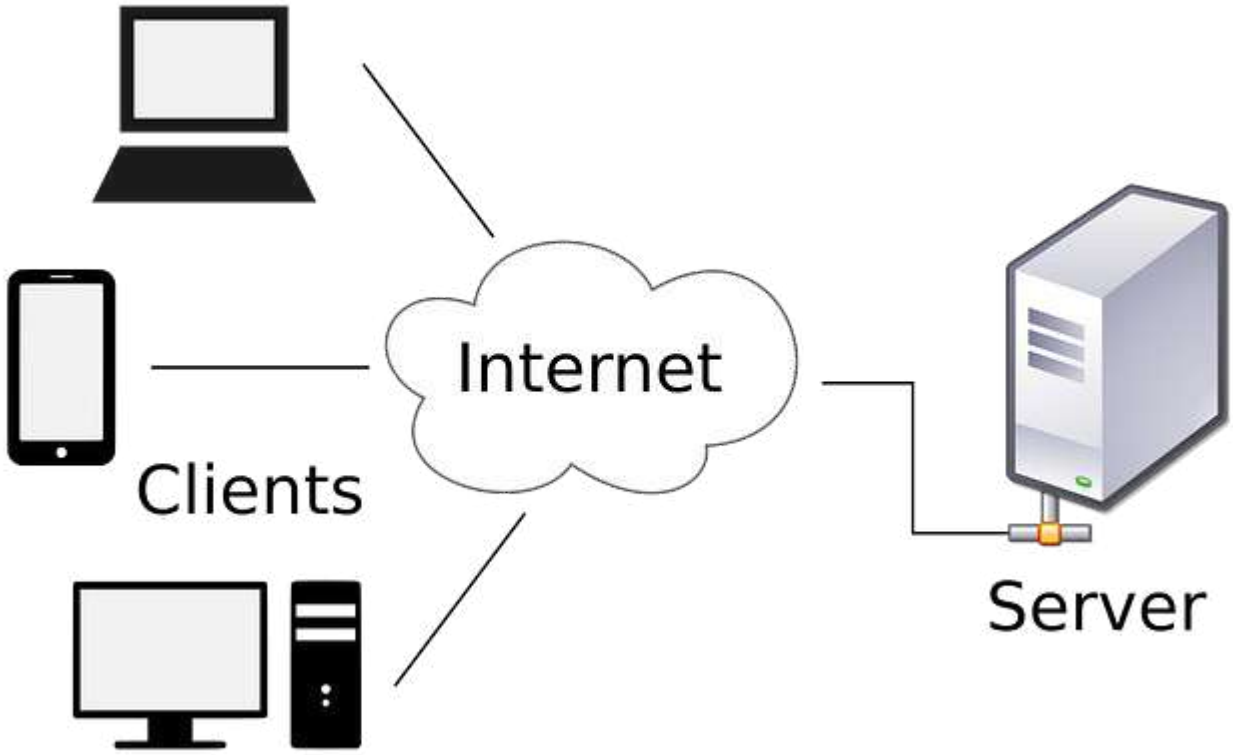
- Onde cada parte desse sistema realmente está?

Conceito fundamental

- A Web funciona com um modelo de computação chamado:
  - Cliente-Servidor

- É uma forma de organizar sistemas em que tarefas são divididas entre dois lados.

Definição



Cliente–Servidor é uma arquitetura onde:
O **cliente** solicita serviços
O **servidor** fornece serviços
Eles se comunicam através de uma rede.

O que é o cliente?

- O cliente é o programa que:
  - Inicia a comunicação
  - Faz pedidos
  - Mostra resultados ao usuário

- Na Web, o cliente é:
  - **O navegador**

O que é o servidor?

- O servidor é um computador que:
  - Fica ligado continuamente
  - Recebe pedidos de vários clientes
  - Processa solicitações
  - Envia respostas

A comunicação

- O cliente e o servidor trocam mensagens:
  - Cliente envia um pedido
  - Servidor processa
  - Servidor envia resposta
  - Cliente interpreta

- Isso acontece **toda vez que você clica**.

O que o servidor envia

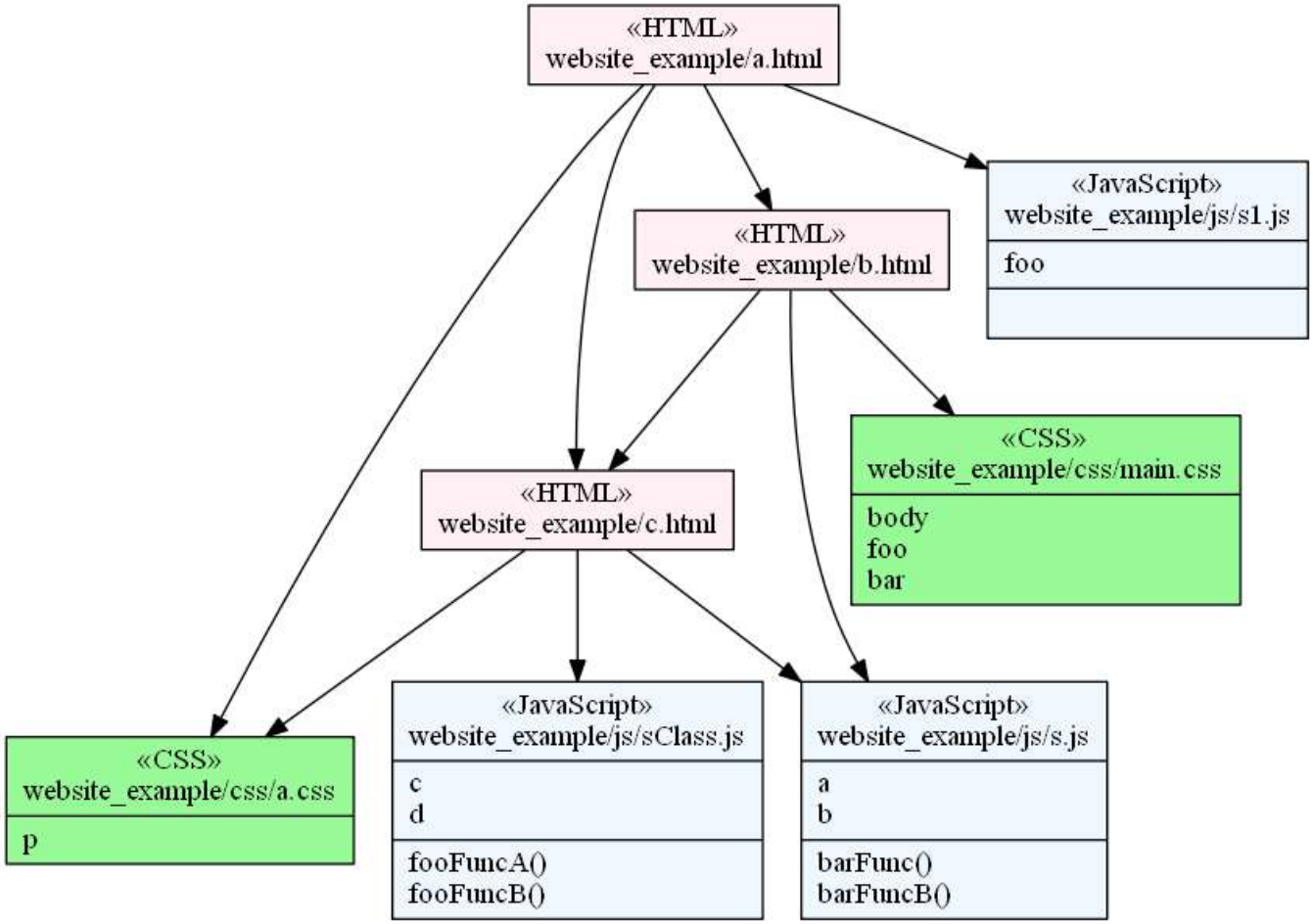
- Na Web, o servidor envia principalmente:
  - Arquivos HTML
  - Arquivos CSS
  - Arquivos JavaScript
  - Dados

O papel do navegador

- O navegador:
  - Recebe os arquivos
  - Interpreta o código
  - Constrói a interface
  - Executa o JavaScript

- Ele é um **interpretador de tecnologias Web**.

O trio da Web


«HTML»
website_example/a.html
«JavaScript»
website_example/js/s1.js
foo
«HTML»
website_example/b.html
«CSS»
website_example/css/main.css
body
foo
bar
«HTML»
website_example/c.html
«CSS»
website_example/css/a.css
p
«JavaScript»
website_example/js/sClass.js
c
d
fooFuncA()
fooFuncB()
«JavaScript»
website_example/js/s.js
a
b
barFunc()
barFuncB()

O trio da Web

- A interface WEB é construída com três tecnologias principais:
  - HTML (Estrutura)
  - CSS (Aparência)
  - JavaScript (Comportamento)

HTML

- HTML (HyperText Markup Language):
  - Define a estrutura da página
  - Organiza conteúdos
  - Indica o significado dos elementos

- Sem HTML → não há página.

CSS

- CSS (Cascading Style Sheets):
  - Controla aparência
  - Cores, fontes, espaçamento
  - Layout da página

- Sem CSS → página funciona, mas é “crua”.

JavaScript

- JavaScript:
  - Torna a página interativa
  - Responde a cliques
  - Muda conteúdos dinamicamente
- Sem JS → página é estática.

O que o navegador faz?

- O navegador:
  - Lê HTML
  - Aplica CSS
  - Executa JavaScript
- Ele é um **interpretador de tecnologias Web**.

O trio WEB

- Essas três “linguagens”:
  - Rodam no cliente
  - São interpretadas pelo navegador
  - Formam o front-end

Introdução ao HTML

Revisão dos Conceitos Fundamentais

O que é HTML?

- HTML (HyperText Markup Language) é uma linguagem de marcação usada para estruturar páginas web.

- Não é uma linguagem de programação, pois não executa lógica.

- Utiliza tags para definir elementos na página.

O que são Tags?

- Tags são elementos usados para estruturar um documento HTML.

- Uma tag pode ter uma abertura e um fechamento.

- Exemplo: Isso é um parágrafo

Estrutura básica de um documento HTML

```
<!DOCTYPE html>
<html>
    <head>
        <title>Minha Página</title>
    </head>

<body>
    <h1>Bem-vindo!</h1>
    <p>Este é um parágrafo.</p>
</body>

</html>
```

O que significa <!DOCTYPE html>?

- Define o tipo de documento e a versão do HTML usada.

- Em HTML5, usamos <!DOCTYPE html> para indicar que estamos usando a versão mais recente.

Tags <html>, <head> e <body>

<html>: Define o início do documento HTML.

<head>: Contém informações como título, estilos e metadados.

<body>: Contém o conteúdo visível da página.

O que incluir na tag <head>?

<title>: Define o título da página.

<meta>: Metadados como charset e descrição.

<link>: Importa arquivos CSS.

<script>: Importa JavaScript.

Tags de Cabeçalho

- As tags <h1>, <h2>, <h3>, <h4>, <h5> e <h6> são usadas para definir **títulos e subtítulos** em uma página HTML.
- Elas representam diferentes níveis de cabeçalho, sendo:

  <h1> → Título principal, mais importante

  <h2> → Subtítulo de nível 2

  <h3> → Subtítulo de nível 3

  <h4> → Subtítulo de nível 4

  <h5> → Subtítulo de nível 5

  <h6> → Subtítulo de nível 6, menos importante

Exemplo:

```
<h1>Meu Título</h1>
<h2>Subtítulo</h2>
```

Tags <p> e

<p>: Define parágrafos.

O navegador adiciona automaticamente um espaço antes e depois de cada <p>, para separá-los visualmente.

O conteúdo dentro da tag <p> pode incluir **texto, imagens, links e outras tags**.

: Quebra de linha.

Exemplo:

```
<p>Este é um parágrafo.</p>
Texto antes da quebra.<br>Texto após a quebra.
```

Tag <img>

- Insere imagens na página.
- Propriedades:
  - **src**: URL da imagem.
  - **alt**: Texto alternativo.
  - **width** e **height**: Tamanho da imagem.
- Exemplo:

```
<img src="imagem.jpg" alt="Descrição"
width="200" />
```

Quando a tag não tem fechamento?

- Algumas tags são auto-fechadas, como <img> e <br>.
- Não precisam de uma tag de fechamento.

Exemplo:

```
<img src="imagem.jpg" alt="Descrição" />
<br />
```

Quando a tag não tem fechamento?

- O HTML5 trouxe mais flexibilidade, permitindo que o <br> e <img> sejam escritos com ou sem barra (/).

✅ Ambas as formas são aceitas em HTML5

Diferenças entre HTML e XHTML

Característica	**HTML**	**XHTML**
Sintaxe	Flexível	Estrita
Fechamento de tags	Opcional para algumas tags (, <img>)	**Obrigatório** (, <img />)
Letras das tags	Pode ser maiúscula ou minúscula (<BODY>, <body>)	**Sempre minúscula** (<body>)
Atributos	Valores podem ser omitidos (<input disabled>)	**Deve ser sempre atribuído** (<input disabled="disabled">)
Aninhamento de tags	Pode ser desorganizado	**Precisa ser correto**
Declaração do DOCTYPE	Opcional	**Obrigatório**

Tag <a> e suas propriedades

- Cria links para outras páginas.

- Propriedades:
  - href: URL do destino.
  - target: Define onde abrir o link (_blank, _self).

- Exemplo:

```
<a href="https://exemplo.com"
   target="_blank">Visite o site</a>
```

Listas em HTML

- Em HTML, existem dois tipos principais de listas:
  - Listas não ordenadas (<ul>)
  - Listas ordenadas (<ol>)

- Ambas usam a tag <li> para definir itens da lista.

Lista Não Ordenada (<ul>)

- Exibe os itens sem uma ordem específica, geralmente com marcadores (bolinhas).
- Exemplo:

```
<ul>
   <li>Item 1</li>
   <li>Item 2</li>
   <li>Item 3</li>
</ul>
```

Lista Ordenada (<ol>)

- Exibe os itens em uma sequência numerada.
- Exemplo:

```
<ol>
   <li>Primeiro item</li>
   <li>Segundo item</li>
   <li>Terceiro item</li>
</ol>
```

Listas Aninhadas

- Podemos criar listas dentro de listas.

- Exemplo:

```
<ul>
   <li>Item 1
      <ul>
         <li>Subitem 1.1</li>
         <li>Subitem 1.2</li>
      </ul>
   </li>
<li>Item 2</li>
</ul>
```

Atividade 03 – Flexbox (CSS)

Criar uma página .html que deverá conter uma estrutura com **três cartões de conteúdo**.

Cada **cartão** deve apresentar:

- uma imagem,
- um título,
- um pequeno texto descritivo e
- um botão com a expressão “Saiba mais”.

Os temas sugeridos para os cartões são **Tecnologia**, **Esportes** e **Educação**, podendo ser utilizados textos curtos relacionados a cada assunto.

O contêiner que envolve os cartões deverá utilizar a propriedade **display: flex** para organizar os elementos. Os cartões devem aparecer lado a lado quando a página for visualizada em telas maiores, com espaçamento entre eles. Dentro de cada cartão também é esperado que haja uma organização adequada dos elementos, podendo utilizar Flexbox para alinhar o conteúdo verticalmente ou distribuir melhor os elementos internos.

Além da organização básica do layout, a página deve apresentar um comportamento simples de responsividade.

Caso haja tempo disponível, melhorar a aparência visual dos cartões adicionando cores, bordas, efeitos de hover, etc.

Flexbox e variáveis do CSS

FLEXBOX

Por que aprender Flexbox?

- Flexbox resolve problemas de layout com mais facilidade que o modelo tradicional baseado em floats.

- Ideal para layouts responsivos e alinhamentos verticais/horizontais.

O que é Flexbox?

- **Flexbox** é um sistema de layout unidimensional (linha ou coluna).

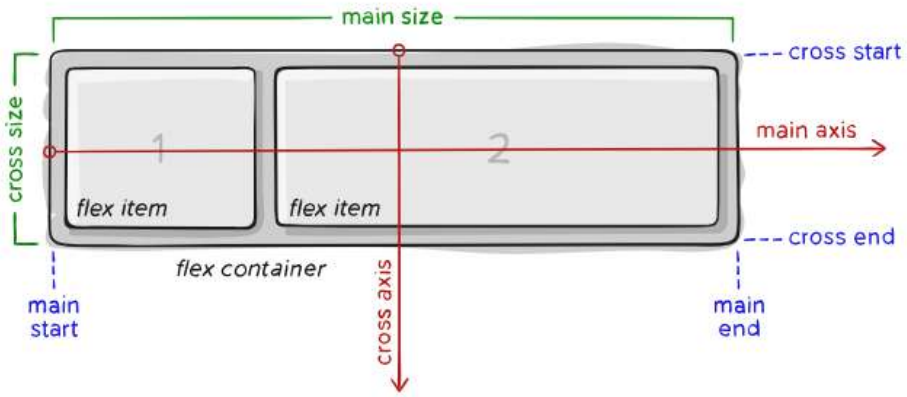
- Altera a largura e a altura dos itens para que o layout se adeque a qualquer tela

- Habilitado:

```
display: flex;
display: inline-flex;
```

Flexbox x Layout “padrão”

- Layout “padrão” é baseado nas direções block e inline
- Flex é baseado em direções “flex flow”
- É um conceito de layout de uma só direção.



Flexbox

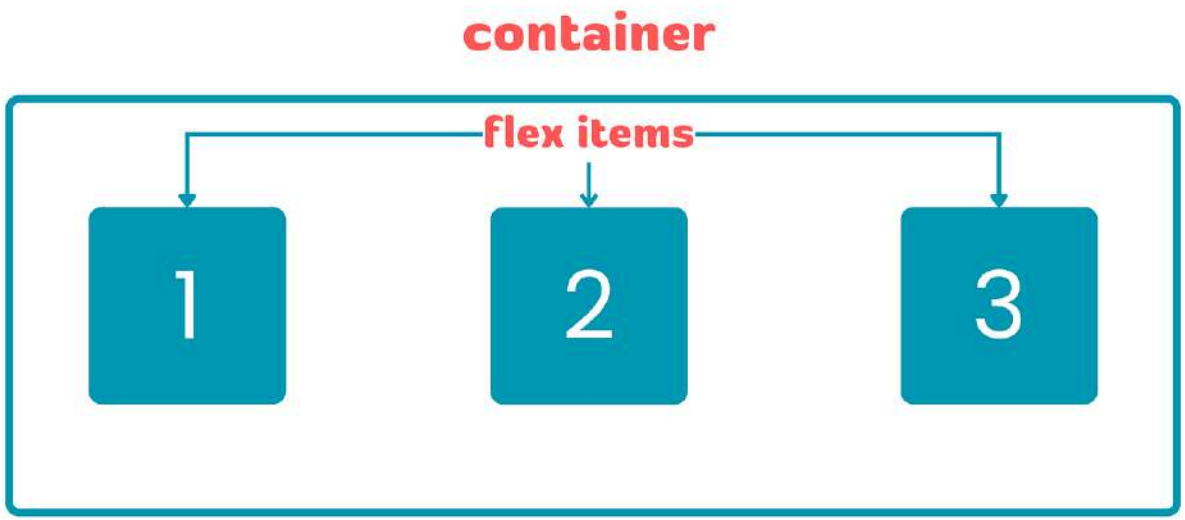
- **Flex container** é o elemento que envolve sua estrutura.

- **Flex itens** são elementos-filhos do flex container.

- **Eixos** são as duas direções básicas que existem em um Flex Container: *main axis*, ou eixo principal, e *cross axis*, ou eixo transversal.

Flexbox

- Flex container é o elemento pai



PROPRIEDADES DO ELEMENTO-PAI

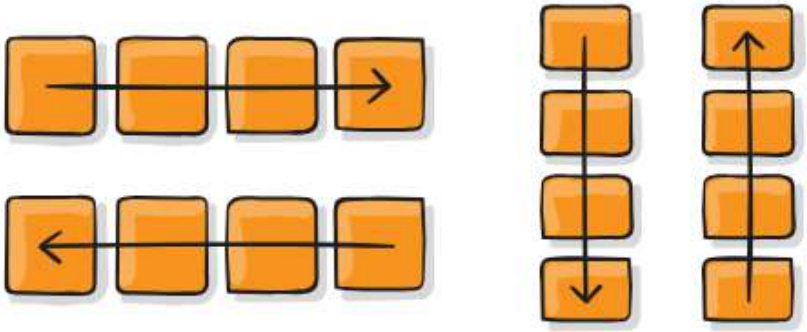
Propriedades do elemento-pai

- **display**: flex | inline-flex
- **flex-direction**: row | row-reverse | column | column-reverse;
- **flex-wrap**: nowrap | wrap | wrap-reverse;
- **flex-flow**: row nowrap | row wrap | column nowrap | column wrap;
- **justify-content**: flex-start | flex-end | center | space-between | space-around | space-evenly;
- **align-items**: stretch | flex-start | flex-end | center | baseline;
- **align-content**: flex-start | flex-end | center | space-between | space-around | stretch;

flex-direction

- Estabelece o eixo principal, definindo assim a direção em que os *flex items* são alinhados no *flex container*.

```
row
row-reverse
column
column-reverse
```

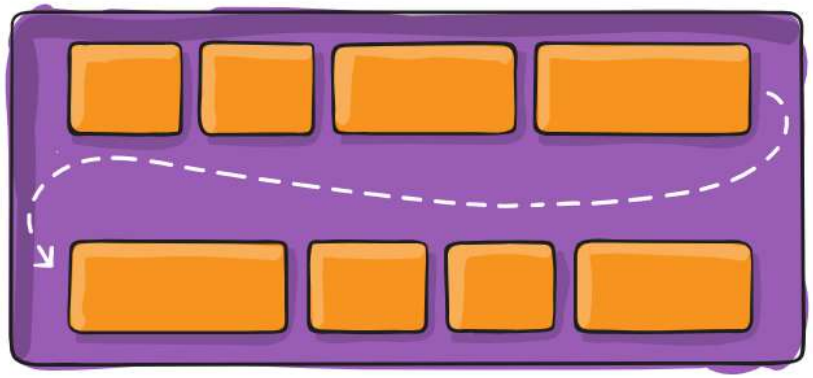


https://encurtador.com.br/PFCrp

flex-wrap

- todos os *flex items* tentam se ajustar em uma **única linha** dentro do *flex container*.
- os itens podem ficar espremidos ou sair do container.
- flex-wrap permite que os *flex items* **quebrem para uma nova linha**, quando necessário.

```
nowrap
wrap
wrap-reverse
```



https://encurtador.com.br/Vme46

flex-flow

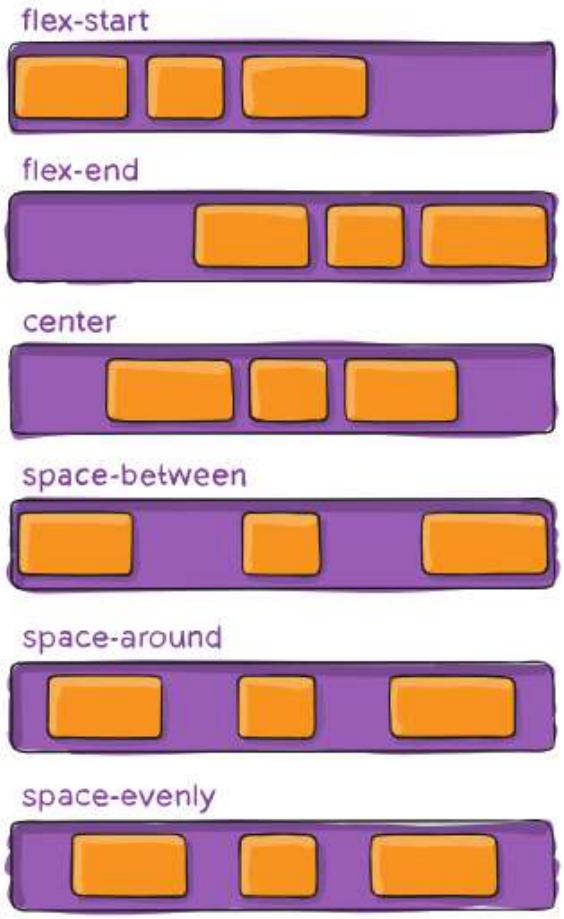
- Determina quais os eixos principal e transversal.
- Permite definir **duas propriedades ao mesmo tempo**: flex-direction e flex-wrap.

https://developer.mozilla.org/pt-BR/docs/Web/CSS/flex-flow

- Exemplo

```
row nowrap
row wrap
column nowrap
column wrap
```

justify-content



- Define o alinhamento dos itens ao longo do eixo principal.
- Ajuda a distribuir o espaço livre que sobrar.

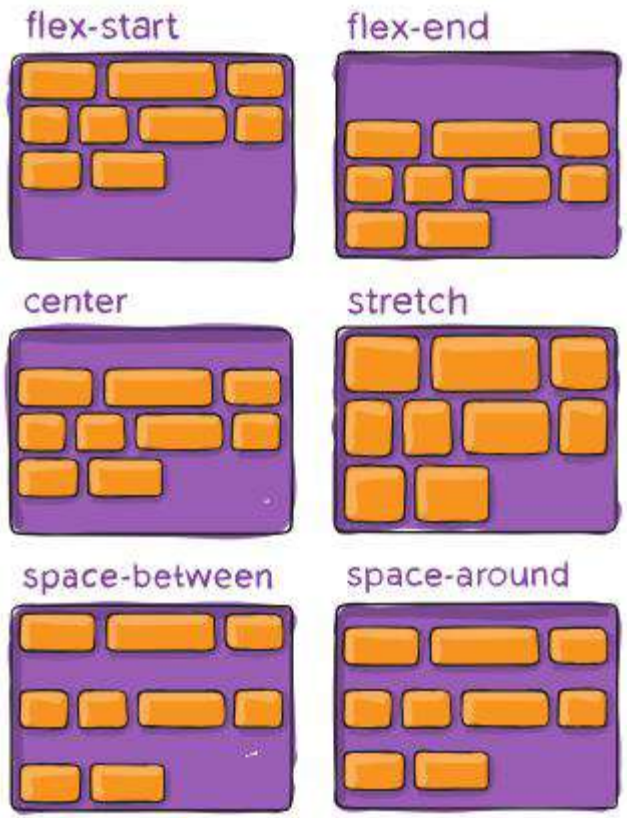
```
flex-start
flex-end
center
space-between
space-around
space-evenly
left
right
```

https://encurtador.com.br/Q4v8H

align-content

- Organiza as linhas dentro de um flex container quando há espaço extra no eixo transversal.

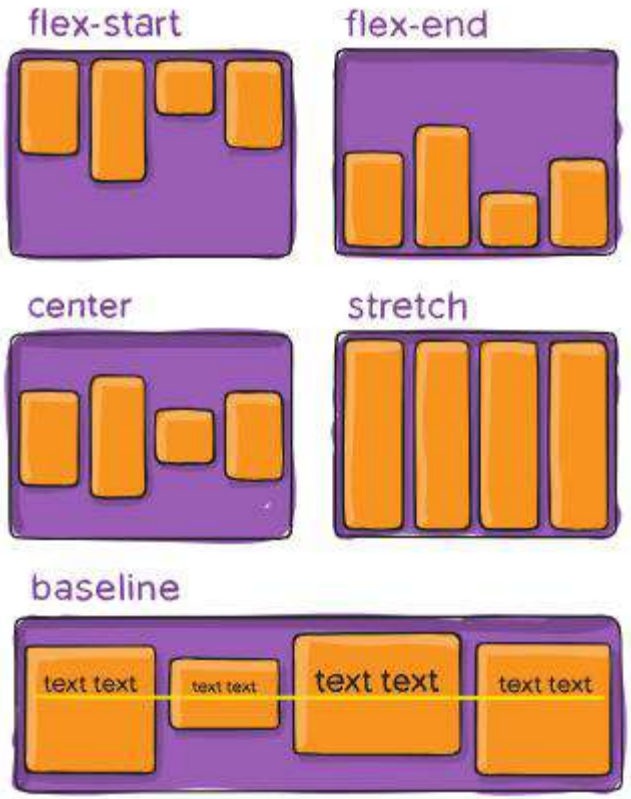
https://encurtador.com.br/dT2la



align-items

- define o comportamento padrão de como *flex items*
- são alinhados de acordo com o eixo transversal

https://encurtador.com.br/piGet



Outras propriedades

- `gap, row-gap e colum-gap`
  - define o **espaçamento entre os itens flexíveis**, sem precisar colocar margin nos filhos!
  - gap: 10px;

- `direction`
  - define a **direção do texto** (não é exclusiva do Flexbox, mas afeta layout).
  - direction: rtl; /* right to left */

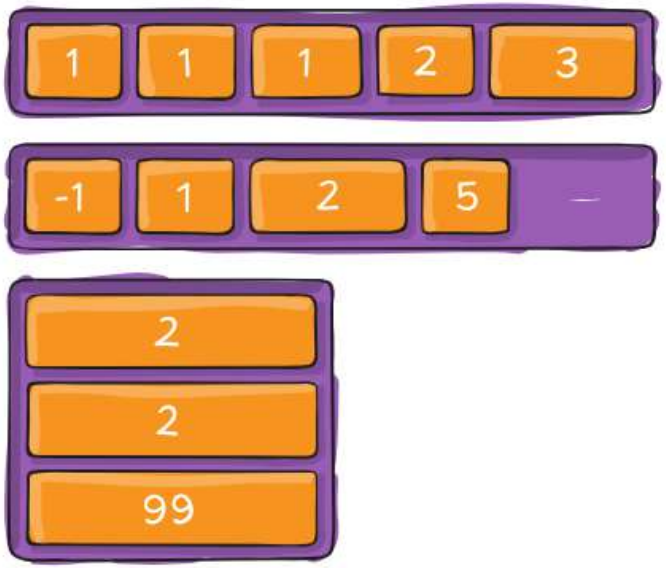
- `place-content, place-items, place-self`
  - combina alinhamentos verticais e horizontais:
  - place-content: center space-between; /* atalho para align-content + justify-content */
  - place-items: center start; /* atalho para align-items + justify-items */

PROPRIEDADE DOS ELEMENTOS-FILHOS

order

- Determina a ordem em que os elementos aparecerão.

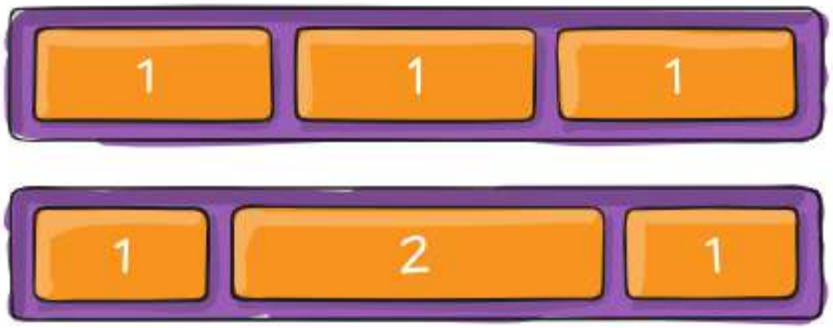
- Por padrão os flex items são dispostos na tela na ordem do código.



https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Ordering_flex_items#the_order_property

flex-grow

- Define a habilidade de um flex item de crescer, caso necessário.
- O valor dita a quantidade de espaço disponível no container que será ocupado pelo item.
- :1 - o espaço remanescente no container será distribuído de forma igual entre todos.
- :2 -vai ocupar o dobro de espaço no container com relação aos outros.



flex-shrink

- Define a habilidade de um flex item de encolher, caso necessário.

https://developer.mozilla.org/en-US/docs/Web/CSS/flex-shrink

flex-basis

- Define o tamanho padrão para um elemento antes que o espaço remanescente do container seja distribuído.

- Comprimento ou uma palavra-chave.

- auto, content, max-content, min-content e fit-content.

https://www.w3schools.com/CSSref/tryit.php?filename=trycss3_flex-basis

flex

- Combina ***flex-grow***, ***flex-shrink*** e ***flex-basis*** em uma só propriedade

- Padrão: 0 1 auto

```
.item {
  flex: none |
   [ <'flex-grow'>
     <'flex-shrink'>? ||
     <'flex-basis'> ]
}
```

1: O item **pode crescer** para ocupar o espaço disponível.

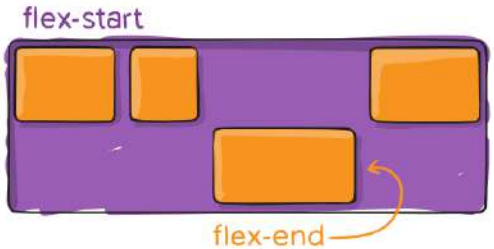
align-self

- Permite que o alinhamento padrão (ou o que estiver definido por align-items) seja sobrescrito para itens individuais.

```
.item {
  align-self: auto | flex-start | flex-end | center
| baseline | stretch;
}
```

https://www.w3schools.com/cssref/tryit.php?filename=trycss3_align-self

https://developer.mozilla.org/en-US/docs/Web/CSS/align-self



Prática

- Complete os 24 desafios do Flexbox Froggy

https://flexboxfroggy.com

VARIÁVEIS DO CSS

Variáveis do CSS

- São formas de **armazenar valores reutilizáveis**, como cores, tamanhos, fontes etc., para facilitar a manutenção e tornar seu código mais organizado e flexível.

- Elas servem para guardar informações — **mas são declaradas e usadas diretamente no CSS**.

Como declarar?

- Normalmente, dentro de um seletor.
- Usualmente, **`:root`**, que representa o elemento <html>
- As variáveis começam com **--**

```
:root {
  --cor-primaria: #3498db;
  --fonte-padrao: 'Arial', sans-serif;
  --espacamento: 16px;
}
```

Como utilizar ?

- A função **`var()`** é usada para acessar o valor da variável.

```
h1 {
  color: var(--cor-primaria);
  font-size: var(--tamanho-fonte-base);
}
```

Valores padrão

```
color: var(--cor-inexistente, #000);
```

Se a variável não existir, a cor será preta.

CSS - Display Grid

Profa. Ariadne Arrais

Por que aprender Grid?

- Grid resolve problemas de layout bidimensional;
- Permite organizar elementos em linhas e colunas;
- Ideal para:
  - Layouts de páginas completas;
  - Interfaces modernas.
- Mais poderoso que float e mais estruturado que flexbox.

Display grid

- É um sistema de layout que organiza elementos em linhas e colunas ao mesmo tempo;
- Controla linhas e colunas simultaneamente;
- Permite o posicionamento preciso dos elementos;
- Diferente do Flexbox:
  - Flexbox: apenas 1 dimensão;
  - Grid: 2 dimensões.

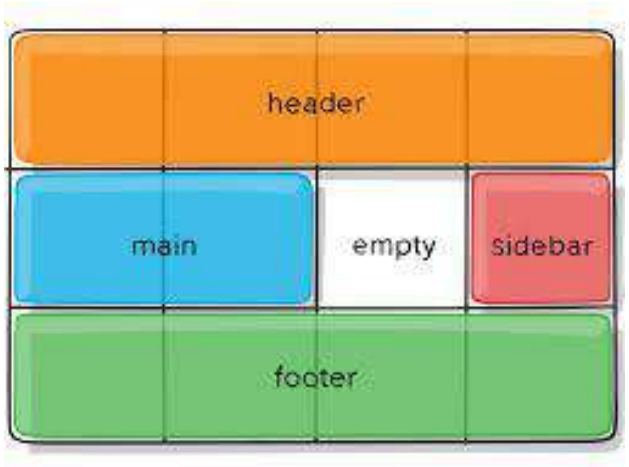
Definindo display do tipo grid

```
display: grid;
display: inline-grid;
```

Grid x Layout “padrão”

- Layout padrão:
  - Baseado em **block** e **inline**.

- Grid:
  - Baseado em **linhas** e **colunas**.



Estrutura do Grid

- Existem dois papéis diferentes:
  - Container (elemento pai)
  - Itens (elementos filhos)

Conceitos principais

- Grid lines
- Grid columns
- Grid rows
- Grid cells
- Grid areas

PROPRIEDADES DO ELEMENTO-PAI

Propriedades do elemento-pai

- Propriedades:
  - display;
  - grid-template-columns;
  - grid-template-rows;
  - gap;
  - grid-auto-rows / grid-auto-columns;
  - justify-self;
  - align-self;
  - place-self.

grid-template-columns

- Especifica o número (e a largura) de colunas em layout do tipo grid.

- Os valores são uma lista separada por espaços, onde cada valor especifica o tamanho da respectiva coluna

```
.grid-container {
        display: grid;
        grid-template-columns: valor1 valor2 valor3;
}
```

https://www.w3schools.com/cssref/tryit.php?filename=trycss_grid-template-columns

grid-template-columns

- **none**: padrão (colunas automáticas)
- **auto**: tamanho baseado no conteúdo/container
- **max-content**: tamanho do maior conteúdo
- **min-content**: tamanho do menor conteúdo
- **length (px, rem...)**: tamanho fixo
- **% (percentual)**: relativo ao container
- **minmax(min, max)**: intervalo de tamanho
- **fit-content()**: ajusta sem ultrapassar o conteúdo
- **repeat()**: repete colunas - **repeat**(12, 1fr);
- **subgrid**: herda grid do pai
- **initial:** valor padrão
- **inherit**: herda do pai

https://www.w3schools.com/cssref/tryit.php?filename=trycss_grid-template-columns

Unidade **fr**

- fr: fração do espaço disponível.
- Divide o espaço restante do container.
- Exemplo:
  - grid-template-columns: 1fr 1fr;
    - Duas colunas de mesmo tamanho.

  - grid-template-columns: 1fr 2fr;
    - Divide em três partes.
    - 1ª colunas – 1 parte; 2ª coluna – 2 partes.
- Considera apenas o espaço livre, primeira respeita os valores fixos (px ou %).

grid-template-rows

- Especifica o número (e a altura) das linhas em um layout do tipo grid.

- Os valores são uma lista separada por espaços, onde cada valor especifica a altura da respectiva linha.

- Exemplo:
  - grid-template-rows: auto auto;
  - grid-template-rows: 50px 100px;
  - grid-template-rows: 20% 20%;
  - grid-template-rows: 70px auto;
  - grid-template-rows: repeat(2, 60px 100px);

https://www.w3schools.com/cssref/playdemo.php?filename=playcss_grid-template-rows

gap

- Define o tamanho do espaçamento entre as linhas e entre as colunas em layouts flexbox, grid ou multicolunas.

- É a abreviação simples das propriedades:
  - row-gap
  - column-gap

https://www.w3schools.com/cssref/playdemo.php?filename=playcss_gap

grid-auto-rows / grid-auto-columns

- Define o tamanho das linhas em um container do tipo grid. Esta propriedade afeta apenas as linhas cujo tamanho não está definido.

justify-self

- Alinha toda a grade ao longo do eixo horizontal dentro de seu contêiner, desde que haja espaço extra não consumido pelas próprias faixas da grade.

- Valores:
  - start; end; center;
  - space-between; space-around; space-evenly;
  - stretch - estica as colunas do grid (especificamente as faixas com tamanho automático) para ocupar todo o espaço restante no container.

align-self

- Especifica o alinhamento para itens dentro de um container do tipo grid ou flexbox.

- Os itens do grid são alinhados na direção do bloco (de cima para baixo e da direita para esquerda).

- Valores: strech, start, end, center, baseline.

place-self

- Alinhas itens individuais do grid e é uma abreviação para as seguintes propriedades:
  - align-self
  - justify-self

https://www.w3schools.com/cssref/playdemo.php?filename=playcss_place-self

PROPRIEDADES DOS ELEMENTOS-FILHOS

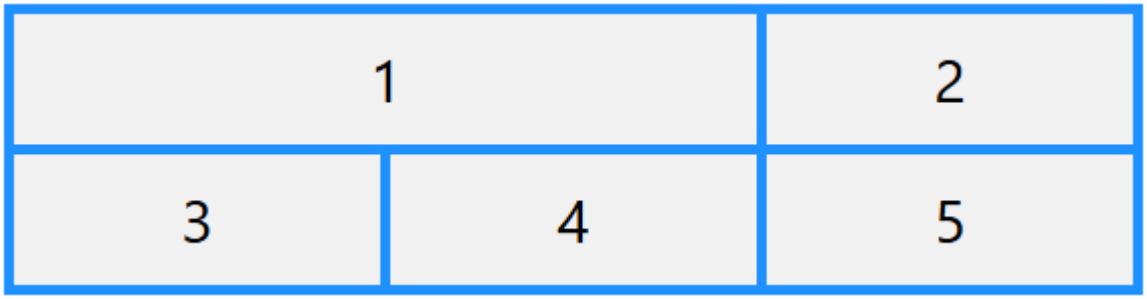
Propriedades

- **grid-column-start**: Especifica em qual coluna o item do grid começará.
- **grid-column-end:** Especifica em qual coluna o item do grid terminará.
- **grid-column:** Atalho para as propriedade *grid-column-start* e *grid-column-end.*
- **grid-row-start:** Especifica em qual linha o item do grid começará.
- **grid-row-end:** Especifica em qual linha o item do grid terminará.
- **grid-row:** Atalho para as propriedade *grid-row-start* e *grid-row-end.*

grid-column

- Define colunas ocupadas

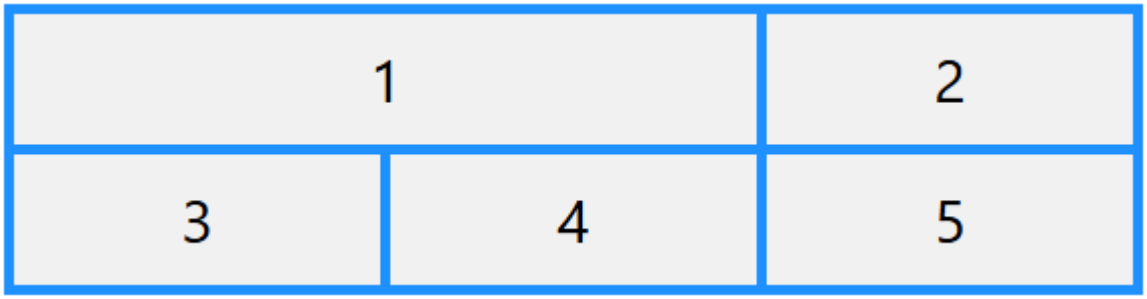
```
.item1 {
  grid-column-start: 1;
  grid-column-end: 3;
}
```



grid-column

- Define colunas ocupadas

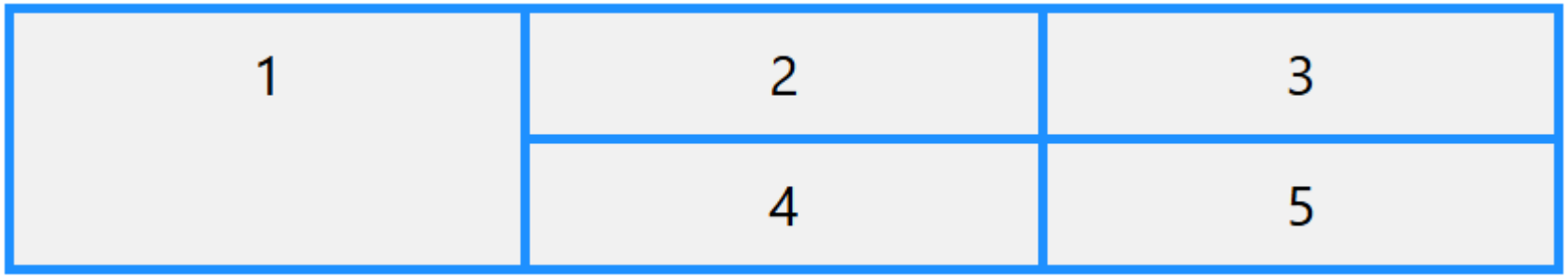
```
.item1 {
  grid-column: 1 / span 2;
}
```



grid-row

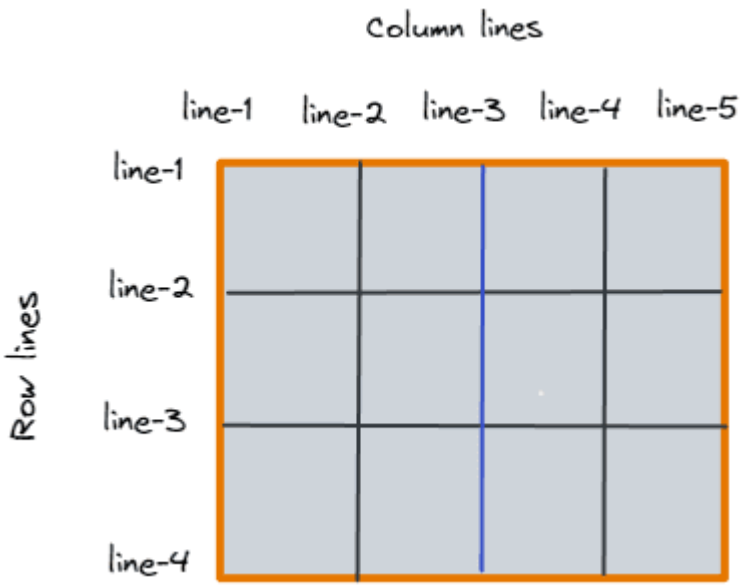
- Define as linhas ocupadas

```
.item1 {
  grid-row-start: 1;
  grid-row-end: 3;
}
```



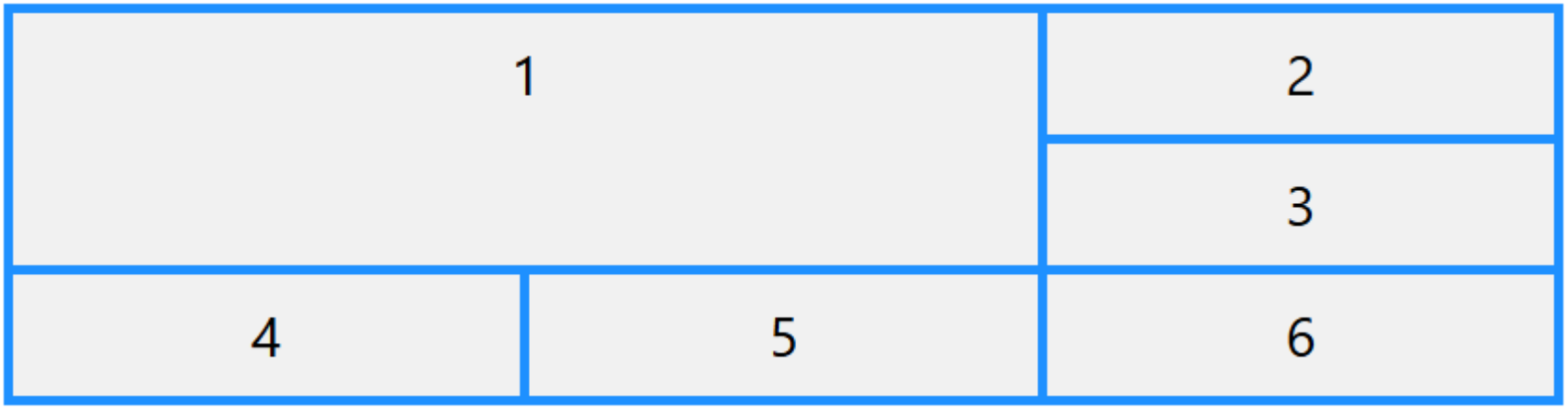
Combinando grid-column e grid-row

- Define linhas ocupadas
  - grid-row: span 2;



grid-area

```
.item1 {
  grid-column: 1 / span 2;
  grid-row: 1 / span 2;
}
```



grid-area

- Atalho para as propriedade *grid-row-start, grid-column-start*, *grid-row-end* e the *grid-column-end*

- grid-area: *grid-row-start / grid-column-start / grid-row-end / grid-column-end* | *itemname*;

https://www.w3schools.com/cssref/tryit.php?filename=trycss_grid-area

GRID TEMPLATES AREAS

Grid templates areas

- Responsável por definir o layout visual.
- Uma propriedade que permite definir uma estrutura de layout nomeada diretamente em CSS, atribuindo nomes às áreas da grade.
- Cada sequência de caracteres representa uma linha, e cada palavra nessa sequência corresponde a uma coluna, possibilitando layouts intuitivos, visuais e fáceis de manter, através da repetição de nomes para abranger as células.

Grid templates areas

```
grid-template-columns: 200px 1fr;
grid-template-rows: auto 1fr auto;
grid-template-areas: "header header"
                     "sidebar main"
                     "footer footer";
```

Prática

- https://cssgridgarden.com/

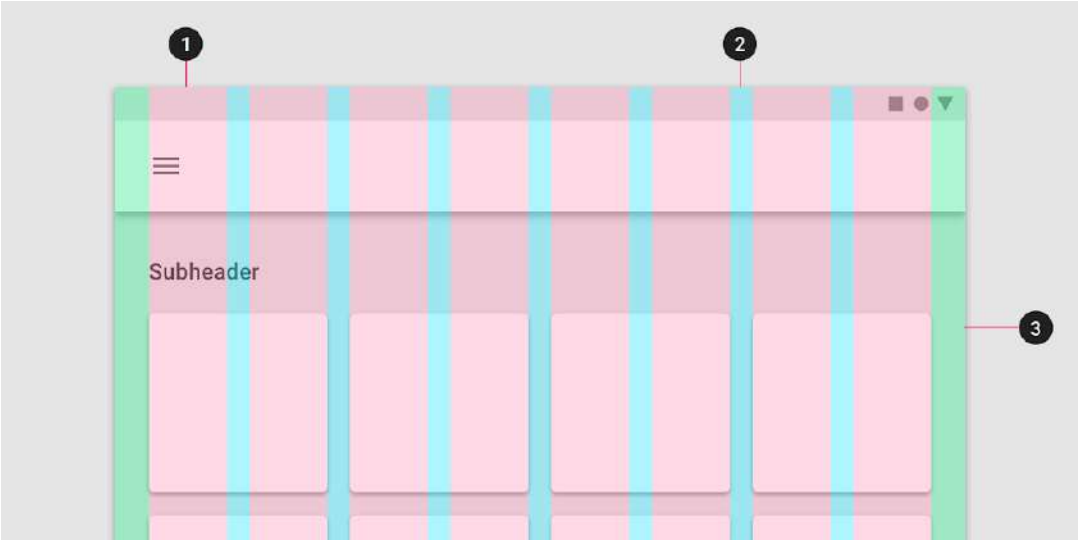
SISTEMA DE 12 COLUNAS

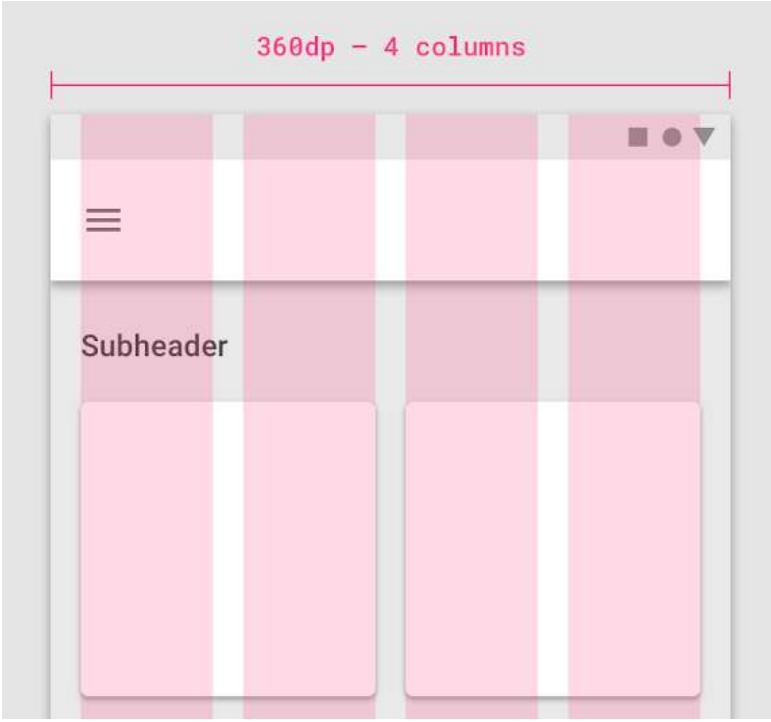
Sistema de 12 colunas

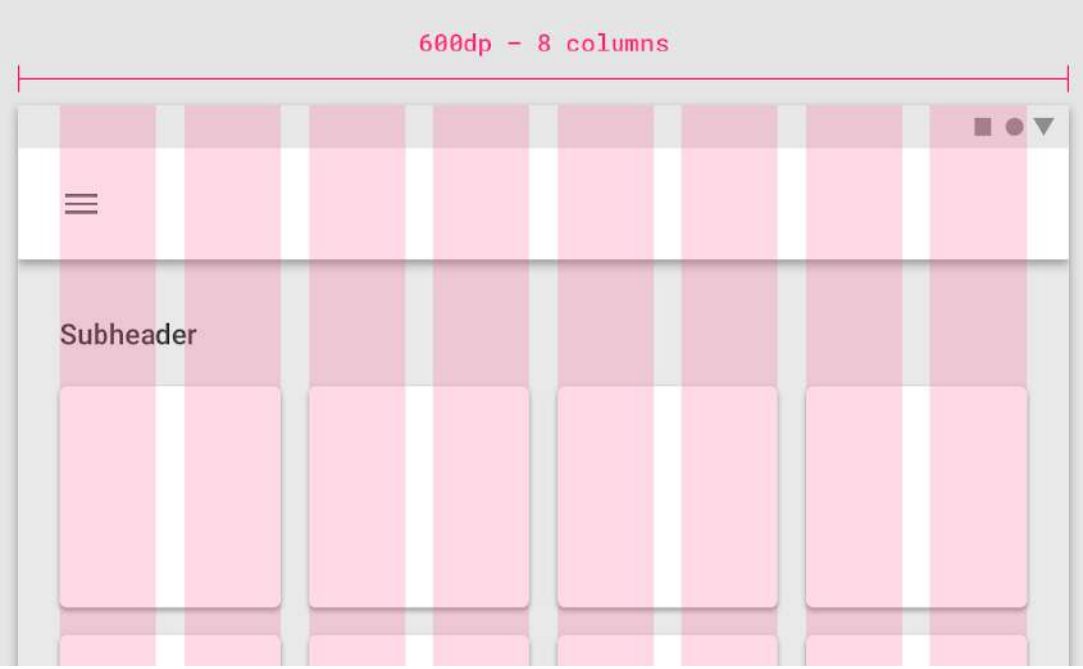
- Um modelo de layout que divide a tela em 12 colunas iguais;
- Usado para organizar o conteúdo de forma proporcional e alinhada, especialmente para layouts responsivos.
- Divide o espaço horizontal disponível em 12 colunas de igual largura, permitindo o posicionamento e dimensionamento precisos dos elementos dentro do layout.

Vantagens

- Tornou-se uma escolha amplamente utilizada devido:
  - Flexibilidade: colunas podem ser combinadas ou redimensionadas;
  - Responsividade: adaptam-se a diferentes tamanhos de tela;
  - Eficiência: estrutura predefinida.


1
2
3
Subheader


360dp – 4 columns
Subheader


600dp – 8 columns
Subheader

Exemplo

```
.container {
display: grid;
grid-template-columns:
repeat(12, 1fr);
gap: 10px;
padding: 10px;
}

.header {
grid-column: span 12;
background: #4CAF50;
padding: 20px;
}

.
```

```
.sidebar {
grid-column: span 3;
background: #2196F3;
padding: 20px;
}

.content {
grid-column: span 9;
background: #FFC107;
padding: 20px;
}

.footer {
grid-column: span 12;
background: #f44336;
padding: 20px;
}
```

RESPONSIVIDADE

Responsividade

- Capacidade do layout se adaptar a diferentes tamanhos de tela.

- O mesmo site precisa funcionar bem em qualquer tela.

Media Queries

- Permitem aplicar CSS dependendo do tamanho da tela.

```
@media (condição) {
        /* regras CSS */
}
```

Layout

- Ao redimensionar um layout para diferentes tamanhos, o grid responsiva ajusta as margens e as larguras do corpo, bem como o número de colunas.

Screen size	Margin	Body	Layout columns
Extra-small (phone)			
0-599dp	16dp	Scaling	4
Small (tablet)			
600-904	32dp	Scaling	8
905-1239	Scaling	840dp	12
Medium (laptop)			
1240-1439	200dp	Scaling	12
Large (desktop)			
1440+	Scaling	1040	12

Seletor de atributo

[class^="col-"]: começa com

[class$="col"]: termina com

[class*="col"]: contém

FUNÇÕES IMPORTANTES

minmax()

- Define um intervalo de tamanho.

grid-template-columns: minmax(200px, 1fr);

mínimo: 200px

máximo: ocupa espaço disponível

repeat()

- Repete colunas e linhas automaticamente
- Evita escrever código repetido

```
grid-template-columns: repeat(3, 1fr);
```

auto-fit e auto-fill

- Criam colunas automaticamente conforme o espaço disponível.
- auto-fit: ajusta e ocupa espaço.
- auto-fill: mantém espaços vazios.

grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));

fit-content

- Define o tamanho baseado no conteúdo.
- Limita o crescimento sem quebrar o layout.
- grid-template-columns: fit-content(300px);
  - Cresce até 300px;
  - Mas respeita o conteúdo.

Animação com CSS

Por que usar?

- Melhoram a experiência do usuário.
- Guiam a atenção para elementos importantes.
- Deixam o site mais interativo.

Principais propriedades

- transition
- transform
- @keyframes + animation

TRANSITION

transition

- **Mudanças de valores de outras propriedades CSS de forma suave.**

- São animações reativas a eventos como `:hover`, `:focus`, etc.

- É possível animar **mais de uma propriedade ao mesmo tempo**

Propriedades de Transição

```
transition: [property] [duration] [timing-function] [delay];
```

Propriedade	O que faz	Exemplo
transition-property	Define qual propriedade será animada	transition-property: all;
transition-duration	Tempo da transição	transition-duration: 0.5s;
transition-timing-function	Curva de aceleração da transição	ease, linear, ease-in-out
transition-delay	Tempo para a animação começar	transition-delay: 0.2s;

`transition-timing-function`

- Define o ritmo da transição. Os mais usados:
  - ease → Começa devagar, acelera, e termina devagar
  - linear → Ritmo constante
  - ease-in → Começa devagar e acelera
  - ease-out → Começa rápido e desacelera
  - ease-in-out → Começa e termina devagar
  - cubic-bezier(...) → Ritmo personalizado

`transition-delay`

- Atraso antes da transição começar

- A transição só começa **após um tempo** em que o evento acontece.

TRANSFORM

`transform`

- Permite **aplicar transformações visuais em elementos HTML**, como **mover**, **girar**, **aumentar/diminuir**, **inclinar**, entre outras.

- A propriedade `transform` **não afeta o fluxo do layout.**

Principais funções

- translate(x, y)
- rotate(ângulo)
- scale(x, y)
- skew(x, y)
- matrix(a, b, c, d, e, f)

translate(x,y)

- Move o elemento no eixo x e/ou y

- Exemplo:

```
/* Move 50px pra direita e 20px pra baixo */
transform: translate(50px, 20px);
```

rotate(ângulo)

- Gira o elemento no sentido horário ou anti-horário

- Exemplo:

```
/* Gira 45 graus no sentido horário */
transform: rotate(45deg);

/* Gira 90 graus no sentido anti-horário */
transform: rotate(-90deg);
```

scale(x,y)

- Aumenta o diminui o tamanho do elemento.

- Exemplo:

```
/* Aumenta 20% em largura e altura */
transform: scale(1.2);

/* Mantém a largura, mas reduz a altura pela metade */
transform: scale(1, 0.5);
```

skew(x,y)

- Inclina o elemento nas direções horizontal e/ou vertical.

- Exemplo:

```
/* Inclina 20° no eixo X e 10° no Y */
transform: skew(20deg, 10deg);
```

`matrix(a,b,c,d,e,f)`

- Combina todas as anteriores num único valor (pouco usada diretamente).

@KEYFRAMES + ANIMATION

O que é o @keyframes

- Define os **quadros-chave** de uma animação em CSS.

- É **mudar de estilo ao longo do tempo**.

```
@keyframes nome-da-animacao {
  0%   { /* estilo inicial */ }
  50%  { /* meio da animação */ }
  100% { /* estilo final */ }
}
```

Propriedade `animation`

- Aplica a animação que foi definida com @keyframes.

```
.elemento {
  animation: nome-da-animacao 2s ease-in-out
infinite;
}
```

Propriedades

Propriedade	O que faz?
animation-name	Nome da animação (ligado ao @keyframes)
animation-duration	Duração da animação (ex: 2s, 500ms)
animation-timing-function	Curva de aceleração (ex: ease, linear, ease-in)
animation-delay	Espera antes de começar
animation-iteration-count	Quantas vezes repete (1, infinite, 3, etc.)
animation-direction	Direção da animação (normal, reverse, alternate, etc.)
animation-fill-mode	Mantém o estilo do início ou fim após a animação (forwards, backwards, both, none)

`animation-timing-function`

- linear
- ease (padrão)
- ease-in
- ease-out
- ease-in-out
- steps(n)
- cubic-bezier(...) (personalizado)

animation-direction

- normal: do início ao fim
- reverse: do fim ao início
- alternate: ida e volta (animação alterna)
- alternate-reverse: volta e ida

`animation-fill-mode`

- none: volta ao estilo original (padrão)
- forwards: mantém o estilo do último frame
- backwards: aplica o estilo do frame inicial durante o delay
- both: aplica os dois acima

`animation-iteration-count`

- 1, 2, 3...
- infinite: repete pra sempre

`animation-play-state`

- running: a animação está rodando
- paused: a animação está pausada (pode pausar via hover ou JS)

USANDO ::BEFORE E ::AFTER COM ANIMATION

`::before` e `::after` com `animation`

- As animações podem ser aplicadas aos pseudo-elementos da mesma forma que em elementos normais.

- A chave aqui é combinar ::before ou ::after com as propriedades de animação CSS, como **`@keyframes`**, **`transition`**, e **`transform`**.

CLIP-PATH

Clip-path

- É uma propriedade CSS que **“recorta”** um elemento, fazendo com que **só uma parte dele fique visível**.

- A parte visível é definida por uma **forma geométrica**, como:
  - circle() → círculo
  - ellipse() → elipse
  - inset() → tipo um retângulo com margens internas
  - polygon() → qualquer forma com múltiplos pontos

Animando...

- Pode **animar** os parâmetros do clip-path usando `transition` ou `@keyframes`

ANIMAÇÕES COM :CHECKED E :TARGET

Animações com `:checked`

- O seletor :checked é usado em **inputs do tipo checkbox ou radio**.

- Quando o input está **marcado (checked)**, ele aciona estilos específicos no CSS, permitindo **alterar propriedades visuais** ou **acionar animações** em outros elementos com ~ ou +.

O que você pode animar com `:checked`?

- Abrir/fechar menus
- Expandir cards
- Trocar temas (light/dark)
- Mostrar/esconder seções
- Animações com `transition` ou

Animações com `:target`

- `:target` é um seletor que se ativa **quando o elemento for o destino de um link** com âncora.

- O que dá pra animar?
  - Expandir descrições
  - Mostrar menus dropdown
  - Criar sliders de conteúdo
  - Estilizar elementos temporariamente

Variáveis do CSS

Variáveis do CSS

- São formas de **armazenar valores reutilizáveis**, como cores, tamanhos, fontes etc., para facilitar a manutenção e tornar seu código mais organizado e flexível.

- Elas servem para guardar informações — **mas são declaradas e usadas diretamente no CSS**.

Como declarar?

- Normalmente, dentro de um seletor.
- Usualmente, **`:root`**, que representa o elemento <html>
- As variáveis começam com **--**

```
:root {
  --cor-primaria: #3498db;
  --fonte-padrao: 'Arial', sans-serif;
  --espacamento: 16px;
}
```

Como utilizar ?

- A função **`var()`** é usada para acessar o valor da variável.

```
h1 {
  color: var(--cor-primaria);
  font-size: var(--tamanho-fonte-base);
}
```

Valores padrão

```
color: var(--cor-inexistente, #000);
```

Se a variável não existir, a cor será preta.

Pseudo-classes em CSS

1. Pseudo-classes de Interação do Usuário

:hover — Quando o mouse passa por cima do elemento.

:focus — Quando o elemento recebe foco (ex: clique ou Tab).

:active — Enquanto o elemento está sendo clicado.

:focus-visible — Como :focus, mas só aparece quando o foco vem do teclado.

:focus-within — Quando o elemento ou algum filho dele está em foco.

2. Pseudo-classes Estruturais (posição no DOM)

:first-child — Seleciona o primeiro filho de um elemento pai.

:last-child — Seleciona o último filho de um elemento pai.

:only-child — Seleciona um elemento que é filho único.

:nth-child(n) — Seleciona o n-ésimo filho (ex: 2n, 3n+1).

:nth-of-type(n) — Seleciona o n-ésimo elemento do mesmo tipo.

:first-of-type — Seleciona o primeiro do tipo entre os irmãos.

:last-of-type — Seleciona o último do tipo entre os irmãos.

:only-of-type — Seleciona o único do seu tipo entre os irmãos.

3. Pseudo-classes de Formulários e Validação

:checked — Seleciona elementos marcados (checkbox ou radio).

:disabled — Seleciona campos desabilitados.

:enabled — Seleciona campos habilitados.

:required — Seleciona campos obrigatórios.

:optional — Seleciona campos opcionais.

:valid — Seleciona campos com valores válidos.

:invalid — Seleciona campos com valores inválidos.

:in-range — Valor dentro de um intervalo (min/max).

:out-of-range — Valor fora do intervalo.

4. Pseudo-classes de Filtragem e Negação

:not(selector) — Seleciona tudo que não corresponde ao seletor.

5. Pseudo-classes Modernas

:is() — Agrupa múltiplos seletores com alta legibilidade.

:where() — Igual ao :is(), mas com zero especificidade.

:has() — Seleciona elemento que contém outro — seletor pai (suporte limitado).

Introdução ao CSS

Estilizando a Web com Elegância

O que é CSS?

- CSS (Cascading Style Sheets)
- Define a aparência e o layout de páginas HTML
- Facilita manutenção e reutilização de estilos
- CSS separa o conteúdo (HTML) da apresentação (estilo)

Como utilizar CSS?

- Inline:

  style="color: blue;"

- Interno:

  <style> dentro do <head>

- Externo:

  Arquivo .css separado (recomendado)

```
body {
   background-color: lightblue;
}
```

Seletores CSS

- Seletores básicos:

  tag
  .classe
  #id

- Seletores avançados:

  div > p
  div p
  p:first-child

- Pseudoclasses e pseudo-elementos

Seletor de tipo (tag)

- Seleciona todos os elementos de um determina tipo (tag HTML).

```
p {
  color: blue;
  font-size: 18px;
}
```

- Todos os <p> da página terão texto azul e tamanho de 18px.

Seletor de classe (.)

- Aplica o estilo a qualquer elemento que possua uma classe específica.
- no *css*:

```
.destaque {
  background-color: yellow;
  font-weight: bold;
}
```

- no *html*:

```
<p class="destaque">Este parágrafo
está em destaque.</p>
```

Seletor de id (#)

- Estiliza um único elemento com um identificador exclusivo.
- no *css*:

```
#cabecalho {
  text-align: center;
  font-size: 24px;
}
```

- no *html*:

```
<h1 id="cabecalho">Título da
Página</h1>
```

Seletor universal (*)

- Aplica estilos a **todos os elementos** da página.
- no *css*:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

- Isso remove margens e preenchimentos padrão de todos os elementos.

Seletor de descendente ( )

- Aplica o estilo **somente aos elementos dentro de outro elemento específico**.
- no *css*:

```
div p {
  color: red;
}
```

- no *html*:

```
<div>
  <p>Este parágrafo ficará vermelho.</p>
</div>
<p>Este parágrafo não será afetado.</p>
```

Seletor filho (>)

- Seleciona **somente os filhos diretos** de um elemento.
- no *css*:

```
div > p {
  color: green;
}
```

- no *html*:

```
<div>
  <p>Este ficará verde.</p>
  <section>
    <p>Este NÃO será afetado.</p>
  </section>
</div>
```

Seletor de irmão adjacente (+)

- Estiliza o **primeiro elemento que vem logo depois** de um outro específico.
- no *css*:

```
h2 + p {
  color: purple;
}
```

- no *html*:

```
<h2>Título</h2>
<p>Este parágrafo ficará roxo.</p>
<p>Este não será afetado.</p>
```

Seletor de irmãos gerais (~)

- Estiliza **todos os elementos do mesmo nível que vêm depois** de um outro específico.
- no *css*:

```
h2 ~ p {
  font-style: italic;
}
```

- no *html*:

```
<h2>Título</h2>
<p>Este ficará itálico.</p>
<p>Este também ficará itálico.</p>
```

Pseudoclasses

- Aplicam estilos com base no **estado do elemento**.
- no *css*: (estiliza os itens ímpares da lista.)

```
li:nth-child(odd) {
  background-color: #eee;
}
```

- no *css*: (estiliza os itens ímpares da lista.)

```
button:hover {
  background-color: lightblue;
}
```

Pseudoclasses

- Por exemplo, pode ser usado para:
  - Estilizar um elemento quando um usuário move o mouse sobre ele
  - Estilize links visitados e não visitados de forma diferente
  - Estilize um elemento quando ele receber foco
  - Estilo de elementos de formulário válidos/inválidos/obrigatórios/opcionais

Pseudoelementos

- Estilizam **partes específicas** de um elemento.
- no *css*:

```
p::first-letter {
  font-size: 2em;
  color: red;
}
```

https://www.w3schools.com/html/html_emojis.asp

```
h1::before {
  content: "⍰ ";
}
h1::after {
  content: " ⍰  ";
}
```

Pseudoelementos

- Por exemplo, pode ser usado para:
  - Estilizar a primeira letra ou linha de um elemento
  - Insira conteúdo antes ou depois de um elemento
  - Estilizar os marcadores dos itens da lista
  - Estilizar a viewbox atrás de uma caixa de diálogo

Usando emojis no HTML

- Em content, o caractere Unicode deve ser representado com

  \ seguido do código hexadecimal . Ex: \1F9D0
- A notação ⌚ funciona em HTML, mas não no CSS.

https://www.w3schools.com/html/html_emojis.asp
https://symbl.cc/pt/unicode-table/#basic-latin

Cascateamento

- O CSS funciona seguindo um sistema de **cascateamento**
  - **prioridade**, **especificidade** e **ordem de declaração**.

- Algumas propriedades são herdadas automaticamente, enquanto outras não.

Cascateamento

- Exemplo:
- no *html*:

```
<p id="texto" class="destaque">Texto de
Exemplo</p>
```

- no *css*:

```
p { color: blue; } /* Regra no CSS externo */
.destaque { color: green; } /* Classe */
#texto { color: orange; } /* ID */
p { color: purple !important; } /* Regra com
!important */
```

Especificidade – Como o CSS Decide Qual Regra Usar?

Tipo de Seletor	Peso na Especificidade	Exemplo
Inline (`style=""`)	1000	`<p style="color: red;">`
ID (`#meuId`)	100	`#meuId {}`
Classe (`.minhaClasse`), Pseudoclasse (`:hover`)	10	`.minhaClasse {}`
Tag (`p` , `h1`)	1	`p {}`

A regra com o maior peso será aplicada primeiro.

O !important ignora a hierarquia normal e **força a aplicação de um estilo**, independentemente da especificidade.

Herança

- Propriedades Herdáveis
  - ✔ color
  - ✔ font-family
  - ✔ visibility

- Propriedades NÃO Herdáveis
  - margin, padding, border
  - width, height
  - background, display

Herança

- Se quiser **forçar a herança** em uma propriedade não herdável, use **inherit**:
- no *css*:

```
div {
  background-color: inherit; /* Herda a cor do
pai */
}
```

```
p {
  color: initial; /* Volta ao padrão do
navegador */
}
```

Cascateamento e Herança

- O CSS segue uma ordem de prioridade: Inline > Interno > Externo > Padrão
- A última regra aplicada tem mais peso se os seletores forem iguais.
- IDs são mais fortes que classes, que são mais fortes que tags.
- O !important quebra a hierarquia e força a regra.
- Algumas propriedades são herdadas automaticamente, outras não.
- Dica Final: Sempre prefira uma estrutura organizada de seletores e evite !important sempre que possível.

Cores

- Em CSS, podemos definir cores de várias maneiras (https://htmlcolorcodes.com/).
  - **Nomes de Cores:** Exemplo: red, blue, green.
  - **Hexadecimal:** Exemplo: #FF5733.
  - **RGB (Red, Green, Blue):** Exemplo: rgb(255, 87, 51).
  - **RGBA (Red, Green, Blue, Alfa):** Exemplo: rgba(255, 87, 51, 0.5) (com transparência).
  - **HSL (Hue, Saturation, Lightness):** Exemplo: hsl(9, 100%, 60%).
  - **HSL (Hue, Saturation, Lightness, Alfa**):hsla(9, 100%, 60%, 0.5) (com transparência).

Fontes

- Usamos as propriedades de fontes para controlar o estilo e a aparência do texto.
  - **font-family:** Define o tipo de fonte.
  - **font-size:** Define o tamanho da fonte.
  - **font-weight:** Define a espessura da fonte.
  - **text-align:** Define o alinhamento do texto.

Propriedades de Fontes

- **font-family:** Especifica a fonte a ser usada.
  - Exemplo: "Arial", "Times New Roman", "Verdana".

- **font-size:** Define o tamanho da fonte. Pode ser em pixels (px), em (em), ou porcentagem (%).
  - Exemplo: 16px, 1em, 120%.

Propriedade de Fontes

- **font-weight:** Define a espessura da fonte. Valores comuns: normal, bold, ou valores numéricos (100 a 900).
  - Exemplo: normal, bold, 700.

- **text-align:** Alinha o texto em relação ao seu contêiner. Pode ser left, center, right ou justify.
  - Exemplo: left, center, right, justify.

Background

- O **background** no CSS refere-se ao estilo visual de fundo de um elemento.

- Pode incluir **cores, imagens, tamanhos e posições**, entre outros.

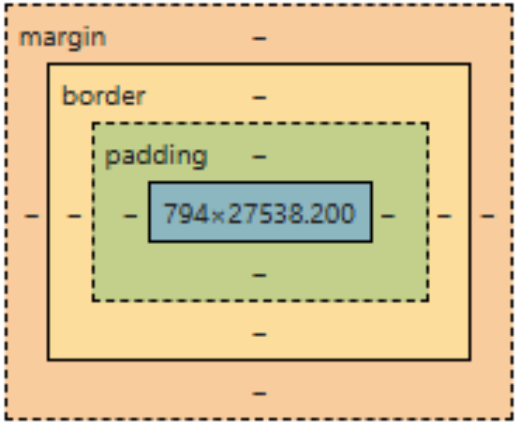
Propriedades do Background

- **background-color**: Define a cor de fundo.
- **background-image**: Define uma imagem de fundo.
- **background-size**: Ajusta o tamanho da imagem de fundo.

Box Model

- O **Box Model** é um conceito importante no layout de páginas web, representando como o conteúdo de um elemento é renderizado.
  - **Content (Conteúdo)**: O conteúdo real do elemento.
  - **Padding (Preenchimento)**: Espaço entre o conteúdo e a borda.
  - **Border (Borda)**: A linha ao redor do elemento.
  - **Margin (Margem)**: Espaço fora da borda, separando o elemento de outros.

Box Model



- **content:** Refere-se ao **conteúdo** real dentro do elemento. O **tamanho do conteúdo** depende do conteúdo textual ou das imagens dentro do elemento.
- **padding: espaçamento entre o conteúdo e a borda** de um elemento.
- **border:** é a linha que envolve o conteúdo e o padding de um elemento.
- **margin: espaçamento externo** ao redor do elemento, separando-o de outros elementos.

Display

- Define como um elemento HTML será exibido na página.

- Ele determina se o elemento ficará em linha (inline), ocupará toda a largura disponível (block), usará um layout flexível (flex), ou se seguirá um sistema de grade (grid), entre outras opções.

Tipos de display

- **block**: Ocupa toda a largura disponível e inicia em uma nova linha.
- **inline**: Ocupa apenas o tamanho do seu conteúdo e permanece na mesma linha.
- **inline-block**: Como inline, mas permite definir altura e largura.
- **flex**: Permite criar layouts flexíveis e responsivos.
- **grid**: Cria um layout baseado em grades.

Tipos de display

- **block**:
  - Sempre inicia em uma nova linha.
  - Ocupa toda a largura disponível.
  - Permite definir altura, largura, margens e paddings.

Tipos de display

- **inline**:
  - Mantém-se na mesma linha de outros elementos inline.
  - Não permite definir altura e largura.

Tipos de display

- **inline-block**:
  - Mantém-se na mesma linha.
  - Permite definir altura e largura.

Tipos de display

- **flex**:
  - Facilita o alinhamento dinâmico dos elementos.
  - Trabalha com espaçamento automático entre itens.

Tipos de display

- **grid**:
  - Divide a página em colunas e linhas.
  - Ideal para layouts complexos.

Tag span

- É um elemento **inline** usado para agrupar partes de um texto ou aplicar estilos a uma parte específica do conteúdo sem afetar o layout ao redor.

- Não tem significado semântico próprio, servindo apenas para estilização ou manipulação via CSS e JavaScript.

Position

- A propriedade position define como um elemento é posicionado na página.

- Existem cinco valores principais:
  - **static** (padrão)
  - **relative**
  - **absolute**
  - **fixed**
  - **sticky**

- Cada um tem um comportamento diferente em relação ao fluxo do documento.

position: static (Padrão)

- O elemento segue o fluxo normal da página, sem alterações de posição.

- O elemento fica na posição normal, sem deslocamento por top, left, right ou bottom.

- Esse é o comportamento padrão de todos os elementos.

position: relative (Relativo à posição original)

- O elemento pode ser deslocado em relação à sua posição original.
- O elemento **ainda ocupa o espaço original no layout**, mas visualmente ele se desloca.
- **top**, **bottom**, **left** e **right** movem o elemento a partir da posição normal.

position: absolute (Fora do fluxo do documento)

- O elemento é removido do fluxo normal e posicionado em relação ao primeiro elemento pai que tenha position: relative, absolute ou fixed.

- Se não houver um ancestral com position: relative, ele será posicionado **em relação ao <html> (viewport).**

- Ele **não ocupa espaço no fluxo normal** do layout.

position: fixed (Fixo na tela, independente do scroll)

- O elemento é fixado na tela e não se move ao rolar a página.

- Ele fica **fixo na tela**, independentemente do rolar da página.

- Muito usado para menus fixos, botões de "voltar ao topo", etc.

position: sticky (Mistura de relative e fixed)

- O elemento se comporta como relative até atingir um determinado ponto da tela, onde então ele se fixa.

- Ele age como um elemento relative até que a página role além do top: 0px, momento em que ele se fixa como fixed.

- Muito usado para **menus fixos ao rolar a página.**

Resumo - position

Tipo	Ocupa espaço normal?	Se move no layout?	Se fixa na tela?
static	☑Sim	✕Não	✕Não
relative	☑Sim	☑Sim (em relação à posição original)	✕Não
absolute	✕Não	☑Sim (em relação ao primeiro ancestral posicionado)	✕Não
fixed	✕Não	☑Sim (em relação à tela)	☑Sim
sticky	☑Sim	☑Sim (até atingir um ponto, onde se fixa)	☑Parcialmente

Dúvidas?

Introdução ao CSS

Estilizando a Web com Elegância

O que é CSS?

- CSS (Cascading Style Sheets)
- Define a aparência e o layout de páginas HTML
- Facilita manutenção e reutilização de estilos
- CSS separa o conteúdo (HTML) da apresentação (estilo)

Como utilizar CSS?

- Inline:

  style="color: blue;"

- Interno:

  <style> dentro do <head>

- Externo:

  Arquivo .css separado (recomendado)

```
body {
   background-color: lightblue;
}
```

Seletores CSS

- Seletores básicos:

  tag
  .classe
  #id

- Seletores avançados:

  div > p
  div p
  p:first-child

- Pseudoclasses e pseudo-elementos

Seletor de tipo (tag)

- Seleciona todos os elementos de um determina tipo (tag HTML).

```
p {
  color: blue;
  font-size: 18px;
}
```

- Todos os <p> da página terão texto azul e tamanho de 18px.

Seletor de classe (.)

- Aplica o estilo a qualquer elemento que possua uma classe específica.
- no *css*:

```
.destaque {
  background-color: yellow;
  font-weight: bold;
}
```

- no *html*:

```
<p class="destaque">Este parágrafo
está em destaque.</p>
```

Seletor de id (#)

- Estiliza um único elemento com um identificador exclusivo.
- no *css*:

```
#cabecalho {
  text-align: center;
  font-size: 24px;
}
```

- no *html*:

```
<h1 id="cabecalho">Título da
Página</h1>
```

Seletor universal (*)

- Aplica estilos a **todos os elementos** da página.
- no *css*:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

- Isso remove margens e preenchimentos padrão de todos os elementos.

Seletor de descendente ( )

- Aplica o estilo **somente aos elementos dentro de outro elemento específico**.
- no *css*:

```
div p {
  color: red;
}
```

- no *html*:

```
<div>
  <p>Este parágrafo ficará vermelho.</p>
</div>
<p>Este parágrafo não será afetado.</p>
```

Seletor filho (>)

- Seleciona **somente os filhos diretos** de um elemento.
- no *css*:

```
div > p {
  color: green;
}
```

- no *html*:

```
<div>
  <p>Este ficará verde.</p>
  <section>
    <p>Este NÃO será afetado.</p>
  </section>
</div>
```

Seletor de irmão adjacente (+)

- Estiliza o **primeiro elemento que vem logo depois** de um outro específico.
- no *css*:

```
h2 + p {
  color: purple;
}
```

- no *html*:

```
<h2>Título</h2>
<p>Este parágrafo ficará roxo.</p>
<p>Este não será afetado.</p>
```

Seletor de irmãos gerais (~)

- Estiliza **todos os elementos do mesmo nível que vêm depois** de um outro específico.
- no *css*:

```
h2 ~ p {
  font-style: italic;
}
```

- no *html*:

```
<h2>Título</h2>
<p>Este ficará itálico.</p>
<p>Este também ficará itálico.</p>
```

Pseudoclasses

- Aplicam estilos com base no **estado do elemento**.
- no *css*: (estiliza os itens ímpares da lista.)

```
li:nth-child(odd) {
  background-color: #eee;
}
```

- no *css*: (estiliza os itens ímpares da lista.)

```
button:hover {
  background-color: lightblue;
}
```

Pseudoclasses

- Por exemplo, pode ser usado para:
  - Estilizar um elemento quando um usuário move o mouse sobre ele
  - Estilize links visitados e não visitados de forma diferente
  - Estilize um elemento quando ele receber foco
  - Estilo de elementos de formulário válidos/inválidos/obrigatórios/opcionais

Pseudoelementos

- Estilizam **partes específicas** de um elemento.
- no *css*:

```
p::first-letter {
  font-size: 2em;
  color: red;
}
```

https://www.w3schools.com/html/html_emojis.asp

```
h1::before {
  content: "⍰ ";
}
h1::after {
  content: " ⍰  ";
}
```

Pseudoelementos

- Por exemplo, pode ser usado para:
  - Estilizar a primeira letra ou linha de um elemento
  - Insira conteúdo antes ou depois de um elemento
  - Estilizar os marcadores dos itens da lista
  - Estilizar a viewbox atrás de uma caixa de diálogo

Usando emojis no HTML

- Em content, o caractere Unicode deve ser representado com

  \ seguido do código hexadecimal . Ex: \1F9D0
- A notação ⌚ funciona em HTML, mas não no CSS.

https://www.w3schools.com/html/html_emojis.asp
https://symbl.cc/pt/unicode-table/#basic-latin

Cascateamento

- O CSS funciona seguindo um sistema de **cascateamento**
  - **prioridade**, **especificidade** e **ordem de declaração**.

- Algumas propriedades são herdadas automaticamente, enquanto outras não.

Cascateamento

- Exemplo:
- no *html*:

```
<p id="texto" class="destaque">Texto de
Exemplo</p>
```

- no *css*:

```
p { color: blue; } /* Regra no CSS externo */
.destaque { color: green; } /* Classe */
#texto { color: orange; } /* ID */
p { color: purple !important; } /* Regra com
!important */
```

Especificidade – Como o CSS Decide Qual Regra Usar?

Tipo de Seletor	Peso na Especificidade	Exemplo
Inline (`style=""`)	1000	`<p style="color: red;">`
ID (`#meuId`)	100	`#meuId {}`
Classe (`.minhaClasse`), Pseudoclasse (`:hover`)	10	`.minhaClasse {}`
Tag (`p` , `h1`)	1	`p {}`

A regra com o maior peso será aplicada primeiro.

O !important ignora a hierarquia normal e **força a aplicação de um estilo**, independentemente da especificidade.

Herança

- Propriedades Herdáveis
  - ✔ color
  - ✔ font-family
  - ✔ visibility

- Propriedades NÃO Herdáveis
  - margin, padding, border
  - width, height
  - background, display

Herança

- Se quiser **forçar a herança** em uma propriedade não herdável, use **inherit**:
- no *css*:

```
div {
  background-color: inherit; /* Herda a cor do
pai */
}
```

```
p {
  color: initial; /* Volta ao padrão do
navegador */
}
```

Cascateamento e Herança

- O CSS segue uma ordem de prioridade: Inline > Interno > Externo > Padrão
- A última regra aplicada tem mais peso se os seletores forem iguais.
- IDs são mais fortes que classes, que são mais fortes que tags.
- O !important quebra a hierarquia e força a regra.
- Algumas propriedades são herdadas automaticamente, outras não.
- Dica Final: Sempre prefira uma estrutura organizada de seletores e evite !important sempre que possível.

Cores

- Em CSS, podemos definir cores de várias maneiras (https://htmlcolorcodes.com/).
  - **Nomes de Cores:** Exemplo: red, blue, green.
  - **Hexadecimal:** Exemplo: #FF5733.
  - **RGB (Red, Green, Blue):** Exemplo: rgb(255, 87, 51).
  - **RGBA (Red, Green, Blue, Alfa):** Exemplo: rgba(255, 87, 51, 0.5) (com transparência).
  - **HSL (Hue, Saturation, Lightness):** Exemplo: hsl(9, 100%, 60%).
  - **HSL (Hue, Saturation, Lightness, Alfa**):hsla(9, 100%, 60%, 0.5) (com transparência).

Fontes

- Usamos as propriedades de fontes para controlar o estilo e a aparência do texto.
  - **font-family:** Define o tipo de fonte.
  - **font-size:** Define o tamanho da fonte.
  - **font-weight:** Define a espessura da fonte.
  - **text-align:** Define o alinhamento do texto.

Propriedades de Fontes

- **font-family:** Especifica a fonte a ser usada.
  - Exemplo: "Arial", "Times New Roman", "Verdana".

- **font-size:** Define o tamanho da fonte. Pode ser em pixels (px), em (em), ou porcentagem (%).
  - Exemplo: 16px, 1em, 120%.

Propriedade de Fontes

- **font-weight:** Define a espessura da fonte. Valores comuns: normal, bold, ou valores numéricos (100 a 900).
  - Exemplo: normal, bold, 700.

- **text-align:** Alinha o texto em relação ao seu contêiner. Pode ser left, center, right ou justify.
  - Exemplo: left, center, right, justify.

Background

- O **background** no CSS refere-se ao estilo visual de fundo de um elemento.

- Pode incluir **cores, imagens, tamanhos e posições**, entre outros.

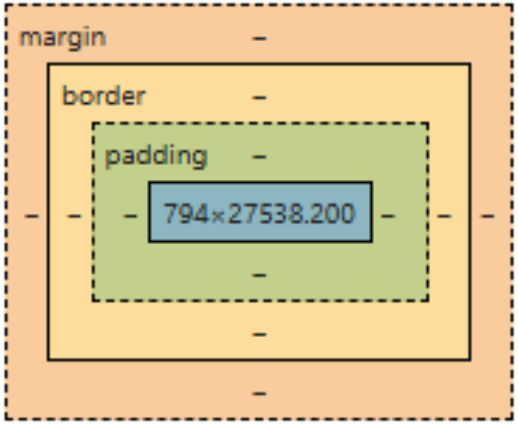
Propriedades do Background

- **background-color**: Define a cor de fundo.
- **background-image**: Define uma imagem de fundo.
- **background-size**: Ajusta o tamanho da imagem de fundo.

Box Model

- O **Box Model** é um conceito importante no layout de páginas web, representando como o conteúdo de um elemento é renderizado.
  - **Content (Conteúdo)**: O conteúdo real do elemento.
  - **Padding (Preenchimento)**: Espaço entre o conteúdo e a borda.
  - **Border (Borda)**: A linha ao redor do elemento.
  - **Margin (Margem)**: Espaço fora da borda, separando o elemento de outros.

Box Model



- **content:** Refere-se ao **conteúdo** real dentro do elemento. O **tamanho do conteúdo** depende do conteúdo textual ou das imagens dentro do elemento.
- **padding: espaçamento entre o conteúdo e a borda** de um elemento.
- **border:** é a linha que envolve o conteúdo e o padding de um elemento.
- **margin: espaçamento externo** ao redor do elemento, separando-o de outros elementos.

Display

- Define como um elemento HTML será exibido na página.

- Ele determina se o elemento ficará em linha (inline), ocupará toda a largura disponível (block), usará um layout flexível (flex), ou se seguirá um sistema de grade (grid), entre outras opções.

Tipos de display

- **block**: Ocupa toda a largura disponível e inicia em uma nova linha.
- **inline**: Ocupa apenas o tamanho do seu conteúdo e permanece na mesma linha.
- **inline-block**: Como inline, mas permite definir altura e largura.
- **flex**: Permite criar layouts flexíveis e responsivos.
- **grid**: Cria um layout baseado em grades.

Tipos de display

- **block**:
  - Sempre inicia em uma nova linha.
  - Ocupa toda a largura disponível.
  - Permite definir altura, largura, margens e paddings.

Tipos de display

- **inline**:
  - Mantém-se na mesma linha de outros elementos inline.
  - Não permite definir altura e largura.

Tipos de display

- **inline-block**:
  - Mantém-se na mesma linha.
  - Permite definir altura e largura.

Tipos de display

- **flex**:
  - Facilita o alinhamento dinâmico dos elementos.
  - Trabalha com espaçamento automático entre itens.

Tipos de display

- **grid**:
  - Divide a página em colunas e linhas.
  - Ideal para layouts complexos.

Tag span

- É um elemento **inline** usado para agrupar partes de um texto ou aplicar estilos a uma parte específica do conteúdo sem afetar o layout ao redor.

- Não tem significado semântico próprio, servindo apenas para estilização ou manipulação via CSS e JavaScript.

Position

- A propriedade position define como um elemento é posicionado na página.

- Existem cinco valores principais:
  - **static** (padrão)
  - **relative**
  - **absolute**
  - **fixed**
  - **sticky**

- Cada um tem um comportamento diferente em relação ao fluxo do documento.

position: static (Padrão)

- O elemento segue o fluxo normal da página, sem alterações de posição.

- O elemento fica na posição normal, sem deslocamento por top, left, right ou bottom.

- Esse é o comportamento padrão de todos os elementos.

position: relative (Relativo à posição original)

- O elemento pode ser deslocado em relação à sua posição original.
- O elemento **ainda ocupa o espaço original no layout**, mas visualmente ele se desloca.
- **top**, **bottom**, **left** e **right** movem o elemento a partir da posição normal.

position: absolute (Fora do fluxo do documento)

- O elemento é removido do fluxo normal e posicionado em relação ao primeiro elemento pai que tenha position: relative, absolute ou fixed.

- Se não houver um ancestral com position: relative, ele será posicionado **em relação ao <html> (viewport).**

- Ele **não ocupa espaço no fluxo normal** do layout.

position: fixed (Fixo na tela, independente do scroll)

- O elemento é fixado na tela e não se move ao rolar a página.

- Ele fica **fixo na tela**, independentemente do rolar da página.

- Muito usado para menus fixos, botões de "voltar ao topo", etc.

position: sticky (Mistura de relative e fixed)

- O elemento se comporta como relative até atingir um determinado ponto da tela, onde então ele se fixa.

- Ele age como um elemento relative até que a página role além do top: 0px, momento em que ele se fixa como fixed.

- Muito usado para **menus fixos ao rolar a página.**

Resumo - position

Tipo	Ocupa espaço normal?	Se move no layout?	Se fixa na tela?
static	☑Sim	✕Não	✕Não
relative	☑Sim	☑Sim (em relação à posição original)	✕Não
absolute	✕Não	☑Sim (em relação ao primeiro ancestral posicionado)	✕Não
fixed	✕Não	☑Sim (em relação à tela)	☑Sim
sticky	☑Sim	☑Sim (até atingir um ponto, onde se fixa)	☑Parcialmente

Dúvidas?